Impact of Preprocessing Choices on Classical NLP Sentiment Analysis

Mudassir Iftikhar Keshav Gupta

Mini Project: IMDB Movie Reviews - Binary Sentiment Classification

Abstract. This paper investigates how five text preprocessing pipelines affect a Logistic Regression classifier on the IMDB movie reviews dataset (50,000 reviews). Using TF-IDF for feature extraction, we vary stopword removal, stemming, and n-gram range. A subsequent grid search over vocabulary size (max_features) and regularisation strength (C) identifies the best configuration: stopword removal with bigrams, max_features=50,000, C=5.0 - achieving 90.85% accuracy and F1 of 0.9090, a 1 pp gain over default settings.

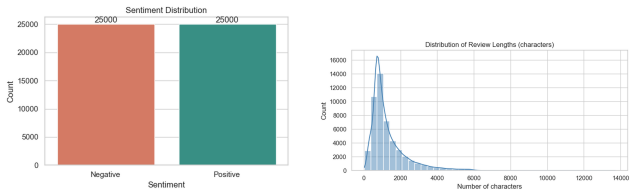
1 Introduction

Sentiment analysis is the task of deciding whether a piece of text is positive or negative. It is one of the most common applications of NLP. While deep learning models get a lot of attention, a simpler approach using TF-IDF features and Logistic Regression still works very well - and has the advantage of being fast, easy to understand, and easy to explain.

One question that often gets overlooked is: does cleaning the text more carefully actually help? In this project, we test five different ways of preprocessing the same data and compare the results. We also tune two important settings, vocabulary size and regularization to see how much improvement we can squeeze out of the model.

2 Dataset and Exploratory Analysis

We used the IMDB Movie Reviews dataset, which contains 50,000 reviews split evenly, 25,000 positive and 25,000 negative. This makes it a perfectly balanced dataset, so there is no bias towards one class. We split the data 80/20: 40,000 reviews for training and 10,000 for testing, keeping the same 50/50 ratio in both splits.



(a) Class balance. (b) Review lengths.

Figure 1: Dataset overview. Classes are perfectly balanced (25,000 each). Review lengths are right-skewed, with most reviews between 500-2,000 characters.

Figure 1 shows the class distribution and review length distribution. The length plot shows most reviews are short-to-medium, but a few are very long, this kind of skew is typical in real-world text data.

3 Methodology

3.1 Preprocessing Pipelines

Every pipeline starts with three common steps: converting text to lowercase, removing punctuation, and splitting into words (tokenization). Each experiment then applies a different combination of extra steps on top, as shown in Table 1.

Table 1: The five experimental preprocessing setups.

Exp	Additional Steps	N-gram
1	None (baseline)	(1,1)
2	Stopword removal	(1,1)
3	Porter stemming	(1,1)
4	Stopwords + stemming	(1,1)
5	Stopwords + bigrams	(1,2)

3.2 Feature Extraction and Classifier

After preprocessing, we convert the text into numbers using TF-IDF, which gives higher scores to words that are important in a review but rare across the whole dataset. We kept the top 10,000 words initially. The classifier is Logistic Regression, a simple but powerful model for binary classification.

3.3 Hyperparameter Tuning

After the main experiments, we ran a grid search on the best pipeline (Exp5) to find the optimal values for two settings: max_features $\in$ {5k, 10k, 20k, 50k} and $C \in \{0.1, 0.5, 1.0, 5.0, 10.0\}$, giving 20 combinations in total.

4 Results

4.1 Preprocessing Experiments

All five experiments achieved over 89% accuracy, which shows that TF-IDF + Logistic Regression is a strong baseline. The full results are in Table 2, and Figures 2 and 3 show the comparison visually.

Table 2: Results across five experiments (sorted by F1).

Experiment	Acc	Prec	Rec	F1
Exp5: SW+Bigrams	0.8985	0.8906	0.9086	0.8995
Exp1: Baseline	0.8968	0.8913	0.9038	0.8975
Exp2: +Stopwords	0.8961	0.8884	0.9060	0.8971
Exp3: +Stemming	0.8947	0.8879	0.9034	0.8956
Exp4: SW+Stem	0.8934	0.8851	0.9042	0.8945

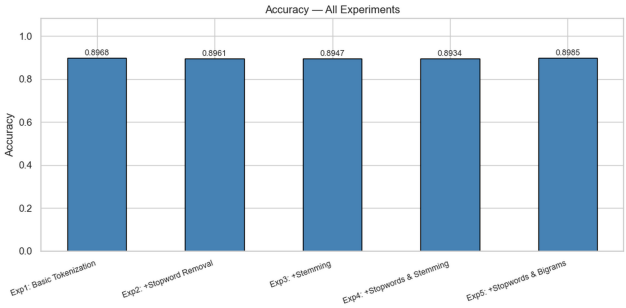


Figure 2: Accuracy across all five experiments.

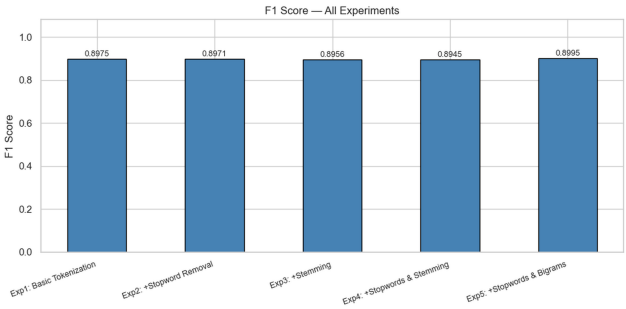


Figure 3: F1-Score across experiments. Exp5 comes first; the two stemming experiments rank lowest.

4.2 Best Model: Confusion Matrix

Figure 4 breaks down the predictions of the best model (Exp5) on the 10,000 test reviews. Out of 5,000 negative reviews, 4,442 were correctly identified and 558 were wrongly predicted as positive. Out of 5,000 positive reviews, 4,543 were correct and 457 were missed. Overall the model makes slightly fewer mistakes on positive reviews than negative ones.

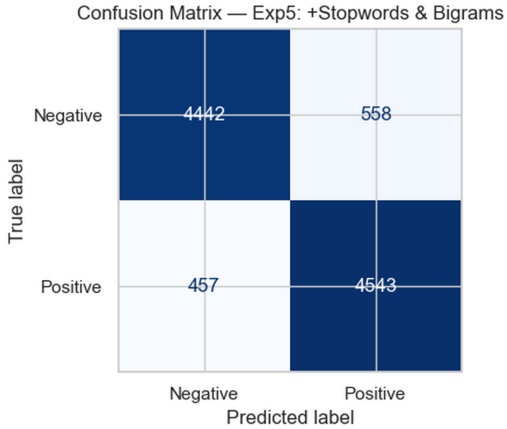


Figure 4: Confusion matrix for Exp5 (best model). TN=4,442 FP=558 FN=457 TP=4,543.

4.3 Hyperparameter Tuning

Figure 5 shows all 20 combinations from the grid search. The darker the cell, the higher the F1 score. The best result came from using 50,000 features and C=5.0, reaching F1=0.9090 and Accuracy=90.85%, about 1% better than the default Exp5 setting.

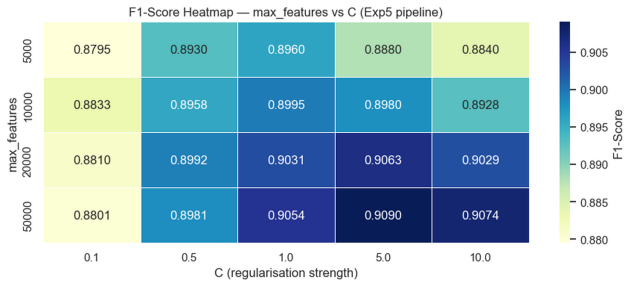


Figure 5: F1-Score heatmap across max_features and C. Darker = higher F1. Best result: 50k features, C=5.0 (F1=0.9090).

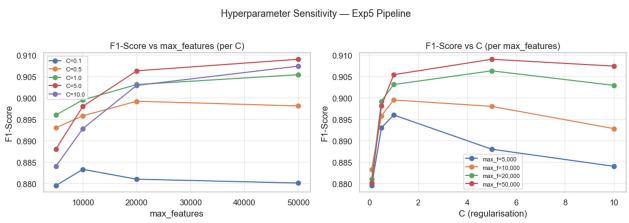


Figure 6: How each parameter affects F1 individually. Left: more features always helps. Right: C=1.0 - 5.0 is the sweet spot.

Table 3 lists the top five combinations from the grid search.

Table 3: Top 5 results from the hyperparameter grid (Exp5 pipeline).

max_feat	C	Acc	F1
50,000	5.0	0.9085	0.9090
50,000	10.0	0.9070	0.9074
20,000	5.0	0.9056	0.9063
20,000	1.0	0.9037	0.9031
20,000	10.0	0.9030	0.9029

5 Discussion

Stemming made things worse. We expected stemming to help by reducing words to their root form. But it actually hurt performance in both Exp3 and Exp4. The reason is that Porter stemming is too aggressive - it treats different words as the same when they are not. For example, "good" and "goods" get the same stem, even though they mean very different things in a review. This caused the model to lose useful information.

Removing stopwords made almost no difference. Stopword removal (Exp2) gave nearly the same result as doing nothing (Exp1). This is because some stopwords like "not" and "never" actually carry important meaning for sentiment. When we remove them, we lose that signal, which cancels out any benefit from reducing noise.

Bigrams gave the biggest improvement. Exp5 was the best of the five preprocessing setups. The reason is that bigrams let the model look at two-word phrases like "not good" or "highly recommended". With only single words (unigrams), the model cannot tell that "not" is negating "good". Bigrams fix this problem.

Bigger vocabulary helped the most during tuning. When we tuned the model, the single biggest improvement came from increasing the vocabulary size from 10,000 to 50,000. This means the model was missing many useful words before. Tuning C also helped a little - the best value was around 1.0–5.0. Too low (C=0.1) and the model is overconstrained; too high (C=10) and it starts to overfit the training data.

6 Conclusion

We tested five different preprocessing pipelines on the IMDB sentiment dataset and found that simple approaches work surprisingly well. The key takeaways are: (1) a basic pipeline with no extra cleaning is already very competitive; (2) stemming hurt performance and should be avoided for sentiment tasks; (3) adding bigrams was the most helpful preprocessing change; (4) increasing vocabulary size to 50,000 was the most impactful tuning step. The final model achieves 90.85% accuracy, trains in under 2 seconds, and requires no complex architecture, showing that classical NLP methods are still very effective when configured carefully.